PART A

(PART A : TO BE REFFERED BY STUDENTS)

Experiment No. 5 (A)

Aim : To implement Lexical Analyzer for given language using Lex tool.

Objective: Develop a program to implement lexical analyzer:

a. Using Lex tool
b. Using finite automata

Outcome: Students are able to design and implement lexical analyzer for given language.

Theory:

The very first phase of compiler is lexical analysis. The lexical analyzer read the input characters and generates a sequence of tokens that are used by parser for syntax analysis. The figure 7.1 summarizes the interaction between lexical analyzer and parser.

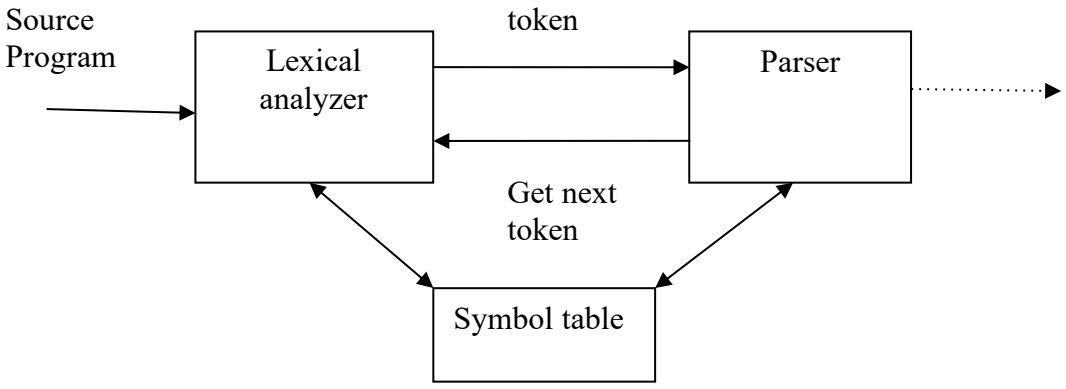


FIGURE: Interaction between lexical analyzer and parser

The lexical analyzer is usually implemented as subroutine or co-routine of the parser. When the "get next token" command received from parser, the lexical analyzers read input characters until it identifies next token.

Lexical analyzer also performs some secondary tasks at the user interface, such as stripping out comments and white spaces in the form of blank, tab and newline characters. It also correlates error messages from compiler to source program. For example lexical analyzer may keep track of number of newline characters and correlate the line number with an error message.

In some compilers, a lexical analyzer may create a copy of source program with error messages marked in it.

An important notation used to specify patterns is a regular expression. Each pattern matches a set of strings. Regular expression will serve as a name for set of strings.

A *recognizer* for a language is a program that takes as input a string *x* and answer "yes" if *x* is a sentence of the language and "no" otherwise.

We compile a regular expression into a recognizer by constructing a generalized transition diagram called a *finite automaton*.

We will design lexical analysis for the language which consists of strings containing “abb” as substring.

Thus, L= {“abb”, “babb”, “abbabba”,…}

The DFA for this language is represented in transition table below:

Now we will simulate this DFA to generate lexical analyzer for the language L.

LEX :

Lex is a program generator designed for lexical processing of character input streams. It accepts a high-level, problem oriented specification for character string matching, and produces a program in a general purpose language which recognizes regular expressions. The regular expressions are specified by the user in the source specifications given to Lex. The Lex written code recognizes these expressions in an input stream and partitions the input stream into strings matching the expressions. At the boundaries between strings program sections provided by the user are executed. The Lex source file associates the regular expressions and the program fragments. As each expression appears in the input to the program written by Lex, the corresponding fragment is executed.

The user supplies the additional code beyond expression matching needed to complete his tasks, possibly including code written by other generators. The program that recognizes the expressions is generated in the general purpose programming language employed for the user's program fragments. Thus, a high level expression language is provided to write the string expressions to be matched while the user's freedom to write actions is unimpaired. This avoids forcing the user who wishes to use a string manipulation language for input analysis to write processing programs in the same and often inappropriate string handling language.

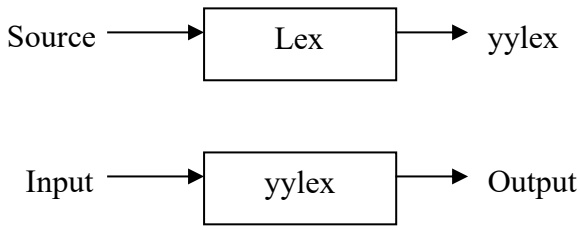


Fig : An overview of Lex

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No.: **C-54**	Name: **Quazi Md Moinuddin**
Class: **TE-C**	Batch: C-
Date of Experiment:**24.03.2025**	Date of Submission: **07.04.2025**
Grade:	

B.1 Software Code written by student:

a) **Using Lex tool**

```
%option noyywrap
%{

%}
%s A B
%%

<INITIAL>1 BEGIN INITIAL;
<INITIAL>0 BEGIN A;
<INITIAL>[^0|\n] BEGIN B;
<INITIAL>\n BEGIN INITIAL; printf("Accepted\n");
<A>1 BEGIN A;
<A>0 BEGIN INITIAL;
<A>[^0|\n] BEGIN B;
<A>\n BEGIN INITIAL; printf("Not Accepted\n");
<B>0 BEGIN B;
<B>1 BEGIN B;
<B>[^0|\n] BEGIN B;
<B>\n {BEGIN INITIAL; printf("INVALID\n");}
%%

void main()
{
yylex();
}
```

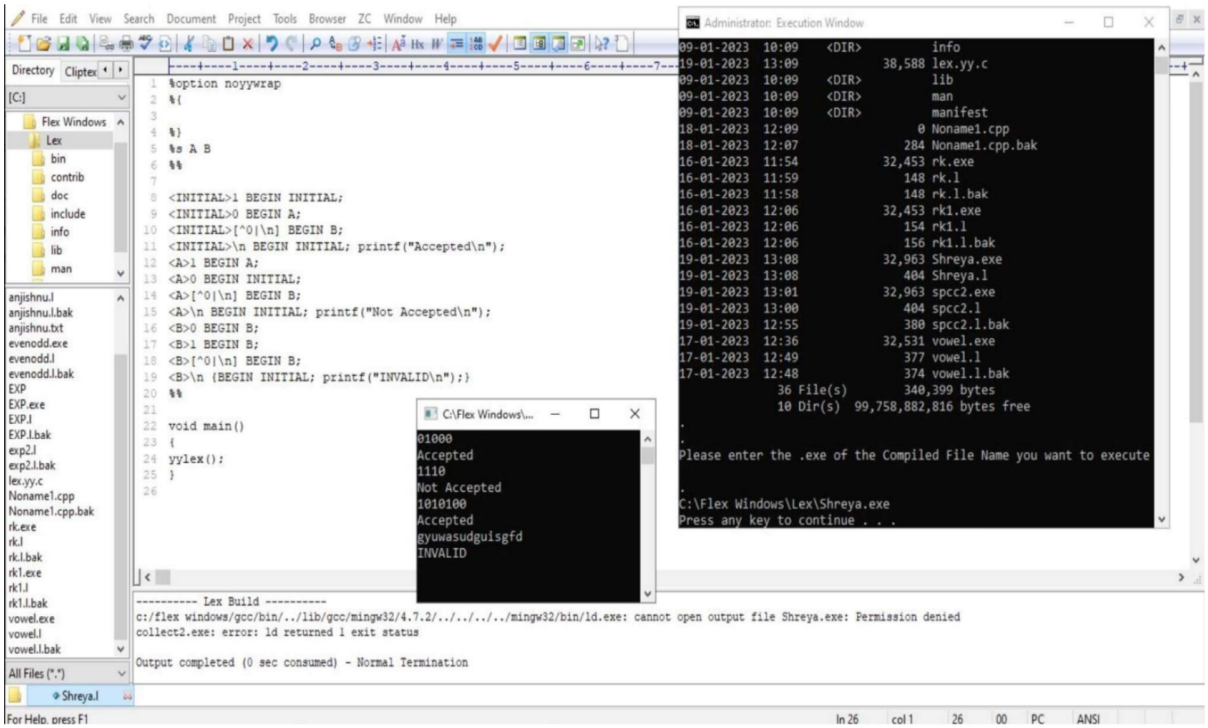
b) Using finite Automata

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
#include <string.h>

void analyze(char str[]);

void main() {
   char input[100];
   int choice;

   clrscr();  // clear screen

   do {
      printf("Enter a line of input to analyze:\n");
      gets(input);  // Read entire line
      analyze(input);

      printf("\nDo you want to analyze another line? (1 = Yes / 0 = No): ");
      scanf("%d", &choice);
      fflush(stdin);  // clear input buffer

      clrscr();  // clear screen for next input

   } while (choice == 1);

   printf("\nThank you for using the Lexical Analyzer!\n");
   getch();  // wait for key press
}

// Function to identify and print tokens
void analyze(char str[]) {
   int i = 0;
   char ch;

   printf("\nLexical Tokens:\n");
   while (str[i] != '\0') {
      ch = str[i];

      if (isspace(ch)) {
         i++;
         continue;
      }

      // Identifier or keyword
      if (isalpha(ch)) {
         char id[20];
         int j = 0;
```

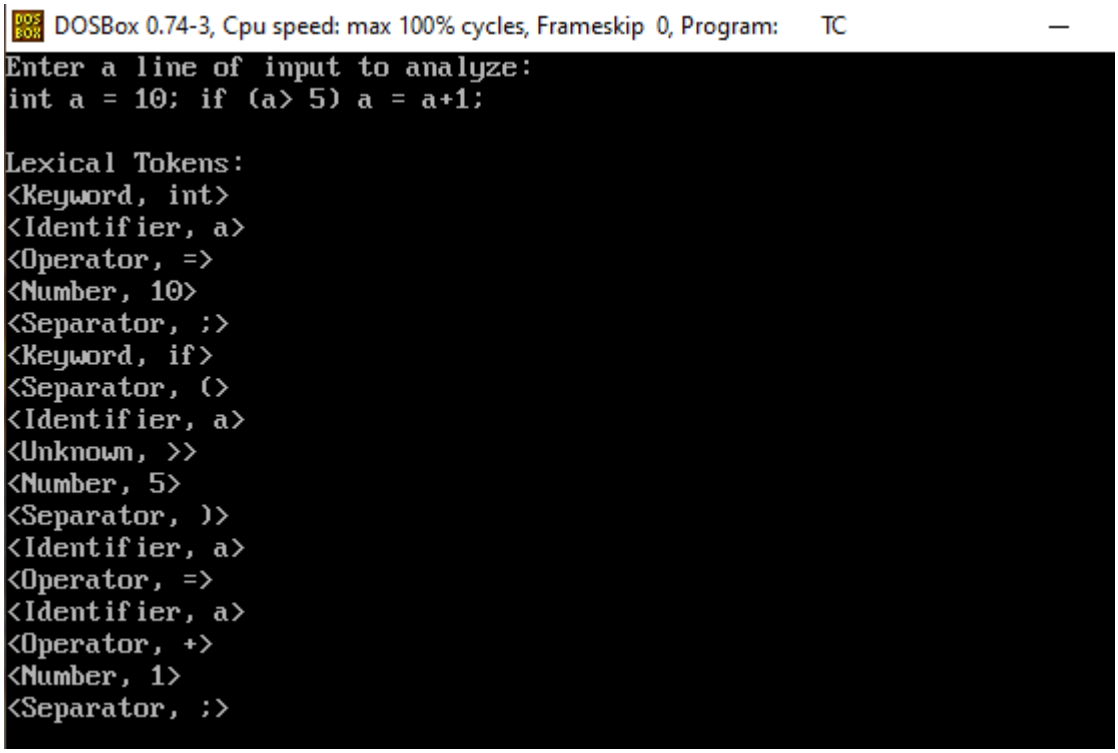
```
            while (isalnum(str[i])) {
                id[j++] = str[i++];
            }
            id[j] = '\0';

            // You can expand this list for more keywords
            if (strcmp(id, "int") == 0 || strcmp(id, "float") == 0 || strcmp(id, "if") == 0 || strcmp(id,
"else") == 0)
                printf("<Keyword, %s>\n", id);
            else
                printf("<Identifier, %s>\n", id);
        }

        // Number
        else if (isdigit(ch)) {
            char num[20];
            int j = 0;

            while (isdigit(str[i])) {
                num[j++] = str[i++];
            }
            num[j] = '\0';
            printf("<Number, %s>\n", num);
        }

        // Operator
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '=') {
            printf("<Operator, %c>\n", ch);
            i++;
        }

        // Separator
        else if (ch == ',' || ch == ';' || ch == '(' || ch == ')' || ch == '{' || ch == '}') {
            printf("<Separator, %c>\n", ch);
            i++;
        }

        // Unknown character
        else {
            printf("<Unknown, %c>\n", ch);
            i++;
        }
    }
}
```

B.2 Input and Output:

a) Using Lex Tool



b) Using finite automata



B.3 Observations and learning:

- Lex simplifies the task of token generation using pattern matching.
- Finite Automata help in understanding the underlying working of lexical analyzers.
- Learnt the difference between keywords, identifiers, operators, and literals.
- Gained hands-on experience in compiler construction phase: lexical analysis.

B.4 Conclusion:

Lexical Analysis is a foundational step in compiler design. It helps in breaking down code into tokens, which are further processed by parsers. The use of Lex tool significantly reduces manual effort, and understanding Finite Automataenhances clarity on how tokens are matched and processed.

B.5 Question of Curiosity

1. What is token?

A **token** is the smallest unit in a programming language that has meaning, such as a keyword, identifier, operator, or symbol.

2. What is the role of lexical analyzer?

The lexical analyzer scans the source code and converts it into tokens by removing whitespaces, comments, and grouping characters into meaningful sequences.

3. What is the output of Lexical analyzer?

The output of a lexical analyzer is a stream of tokens, which is then passed to the syntax analyzer (parser) for further processing.

4. Which errors can be detected by Lexical Analyzer ?

1. Invalid Tokens

- Occurs when an unrecognized sequence of characters is found.
- **Example:** `@var = 5;` → `@` is not a valid character.

2. Unterminated Strings or Characters

- If a string or character literal is not properly closed.
- **Example:** `"hello` → missing closing quote.

3. Invalid Number Formats

- When numbers are written in a wrong format.
- **Example:** `123.45.67` or `09AB` (in decimal context).

4. Illegal Characters

- Use of characters that don't belong to the language character set.
- **Example:** `int π = 3;` → `π` may not be valid in standard C.

5. Length Exceeding Identifier Limits

- Identifiers exceeding the language-specified length.
- **Example:** `int thisVariableNameIsWayTooLongForTheCompiler;`